

NEURAL NETWORKS

Lecture 5 — CS50's Introduction to AI with Python

Lecture Notes Summary

September 3, 2026

Abstract

This document serves as a comprehensive, publication-quality reference summarizing the theoretical foundations, algorithms, and architectures of **Neural Networks** from Lecture 5 of CS50's Introduction to Artificial Intelligence with Python. It covers artificial neurons, activation functions, linear separability, deep networks, backpropagation, convolutional networks for computer vision, and recurrent networks for sequence modeling. All mathematical formulations and concepts are fully grounded in the lecture's source material.

Contents

1	Introduction and Biological Inspiration	2
1.1	Biological Foundations	2
1.2	Artificial Neural Networks (ANN)	2
2	The Artificial Unit (Neuron)	2
2.1	Mathematical Formulation	2
2.2	Primary Activation Functions	3
2.2.1	Step Function	3
2.2.2	Logistic Sigmoid Function	3
2.2.3	Rectified Linear Unit (ReLU)	3
3	Simple Networks and Linear Separability	3
3.1	Logical Operators as Perceptrons	3
3.1.1	The OR Gate	4
3.1.2	The AND Gate	4
3.2	The Linear Separability Constraint	4
4	Multi-Layer Neural Networks and Deep Learning	4
4.1	Hidden Layers	4
4.2	Deep Learning	5
5	Training Neural Networks	5
5.1	Loss Functions	5
5.2	Gradient Descent	5
5.2.1	The Core Algorithm	5
5.2.2	Gradient Descent Variations	5
5.3	Backpropagation	6
5.4	Overfitting and Regularization (Dropout)	6

6	Computer Vision and Convolutional Neural Networks	6
6.1	Digital Image Representation	6
6.2	The Flattening Drawback	7
6.3	Convolutional Neural Networks (CNNs)	7
6.3.1	Image Convolution	7
6.3.2	Pooling	7
6.3.3	The Complete CNN Pipeline	8
7	Recurrent Neural Networks (RNNs)	8
7.1	Feed-Forward vs. Recurrent Networks	8
7.2	Sequence-to-Sequence Relationships	8
7.3	Long Short-Term Memory (LSTM)	9

1 Introduction and Biological Inspiration

Neural networks represent a major paradigm shift in machine learning. Instead of using traditional statistical rules or hand-coded search algorithms, computers are designed to learn complex patterns directly from data.

1.1 Biological Foundations

The architecture of artificial neural networks (ANNs) was inspired as early as the 1940s by neuroscience and studies of how the human biological brain learns:

- **Structure of Neurons:** The brain consists of billions of interconnected cells called **neurons**.
- **Electrical Signaling:** Neurons receive electrical signals from neighboring neurons through physical connections.
- **Threshold Activation:** Individual neurons process these incoming signals and, once they reach a certain electrical threshold, they become activated and propagate further signals to downstream neurons.

1.2 Artificial Neural Networks (ANN)

An **Artificial Neural Network (ANN)** is a mathematical model for machine learning inspired by biological neural networks.

Core Definition: ANN as a Mathematical Function

Mathematically, an artificial neural network is a highly composite function that maps a set of **inputs** to a set of **outputs** based entirely on the structure and parameters (weights and biases) of the network.

The primary objective of training an ANN is to optimize its parameters so that the network can approximate the target function with minimal error.

2 The Artificial Unit (Neuron)

An individual node in an artificial neural network graph is referred to as a **unit** (or artificial neuron).

2.1 Mathematical Formulation

To make a prediction or classification, a unit calculates a **hypothesis function** (h) which is structured in two steps:

1. **Linear Combination:** Calculate the weighted sum of input variables ($x_1, x_2, \dots, x_n$), each multiplied by a corresponding weight ($w_1, w_2, \dots, w_n$), plus an added parameter called the **bias** (w_0):

$$\text{Weighted Sum} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = w_0 + \sum_{i=1}^n w_ix_i \quad (1)$$

The inputs (x_i) represent features of the dataset, and the weights (w_i) reflect the relative importance or influence of each feature.

The term w_0 acts as a **bias** that shifts the value of the function up or down independently of the inputs. In some contexts, it is represented as an additional weight multiplied by a dummy input value of 1.

2. **Activation Function (g):** The linear combination is passed through an activation function g to calculate the final output of the unit:

$$h(x_1, x_2, \dots, x_n) = g \left(w_0 + \sum_{i=1}^n w_i x_i \right) \quad (2)$$

2.2 Primary Activation Functions

Activation functions dictate when and how strongly a unit activates based on its inputs. Three main activation functions are covered in the lecture:

2.2.1 Step Function

The **Step Function** outputs 1 once the weighted input sum meets or exceeds a threshold of 0, and 0 otherwise:

$$g(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

This is the simplest threshold function, commonly used for strict binary classification tasks.

2.2.2 Logistic Sigmoid Function

If the prediction requires real-numbered values representing probabilities rather than a rigid 0-or-1 decision, the **Logistic Sigmoid Function** is used. It produces an S-shaped curve mapping any real-numbered input to a value strictly between 0 and 1:

$$g(x) = \sigma(x) = \frac{1}{1 + e^{-x}} \quad (4)$$

This allows the network to estimate likelihoods (e.g., predicting a 0.8 probability of rain).

2.2.3 Rectified Linear Unit (ReLU)

The **Rectified Linear Unit (ReLU)** is an extremely popular activation function in modern deep learning. It returns the maximum of its input value and 0:

$$g(x) = \max(0, x) \quad (5)$$

If the input is positive, it remains unchanged; if the input is negative, it is leveled out to 0.

3 Simple Networks and Linear Separability

The simplest neural network consists of a single input layer mapped directly to a single output layer containing a single unit. This structure is known as a **Perceptron**.

3.1 Logical Operators as Perceptrons

We can model basic logical gates as simple neural networks using a Step Function as the activation function:

Input x	Input y	Output $(x \vee y)$
0	0	0
1	0	1
0	1	1
1	1	1

Table 1: OR Gate Truth Table

3.1.1 The OR Gate

An OR gate takes two binary inputs (x, y) and outputs 1 if either or both are 1, and 0 only if both are 0.

This can be modeled with weights $w_1 = 1$, $w_2 = 1$, and bias $w_0 = -1$:

- **Input (0, 0):** Weighted sum is $1(0) + 1(0) - 1 = -1 < 0 \implies$ Output = 0.
- **Input (1, 0):** Weighted sum is $1(1) + 1(0) - 1 = 0 \geq 0 \implies$ Output = 1.
- **Input (1, 1):** Weighted sum is $1(1) + 1(1) - 1 = 1 \geq 0 \implies$ Output = 1.

3.1.2 The AND Gate

An AND gate outputs 1 only if both inputs are 1. It can be modeled with the same inputs using weights $w_1 = 1$, $w_2 = 1$, and bias $w_0 = -2$:

- **Input (1, 0):** Weighted sum is $1(1) + 1(0) - 2 = -1 < 0 \implies$ Output = 0.
- **Input (1, 1):** Weighted sum is $1(1) + 1(1) - 2 = 0 \geq 0 \implies$ Output = 1.

3.2 The Linear Separability Constraint

A single-layer perceptron calculates a linear combination of its inputs, meaning its decision boundary is mathematically limited to a straight line (or hyperplane in higher dimensions).

- **Linearly Separable Data:** Data points of different classes that can be cleanly split using a single straight line.
- **Constraint:** A single perceptron is **only capable of learning linearly separable decision boundaries**. It fails completely when confronted with non-linearly separable datasets, such as XOR logical operations or circular target distributions (where one class resides inside a ring and another outside).

4 Multi-Layer Neural Networks and Deep Learning

To bypass the linear separability constraint of single-layer perceptrons, we can combine multiple individual units together into more complex architectures.

4.1 Hidden Layers

A **Multi-Layer Neural Network** is an artificial neural network that introduces one or more intermediate layers between the input layer and the output layer, known as **hidden layers**.

- **Information Flow:** The input variables are passed to the nodes in the first hidden layer. Each hidden unit calculates an intermediate activation value based on its weights and inputs.

- **Feature Combination:** The outputs of the hidden layer then act as the inputs to the subsequent layer (either another hidden layer or the final output layer).
- **Complex Decision Boundaries:** While an individual perceptron can only learn a straight line, each hidden node learns a distinct, independent linear decision boundary. By passing these outputs through subsequent layers, the network can combine these individual lines to form highly complex, non-linear boundaries (such as circles, curves, or polygons).

4.2 Deep Learning

A network containing multiple hidden layers is called a **Deep Neural Network**. Deep learning involves using these multi-layered structures to progressively learn and model higher-level features from raw inputs.

5 Training Neural Networks

Training a neural network involves identifying the optimal set of weights and biases that minimize the model's error on the training data.

5.1 Loss Functions

A **Loss Function** is a mathematical function that quantifies how poorly a given hypothesis models the training data. High loss corresponds to low accuracy, and the goal of training is to minimize this loss value.

5.2 Gradient Descent

Gradient Descent is an optimization algorithm used to minimize loss. It calculates the **gradient** (or slope) of the loss function at the current weight settings, indicating the direction in which the weights should be adjusted to decrease loss.

5.2.1 The Core Algorithm

1. Initialize all weights in the network with small random values. 2. Repeat the following steps to update the weights:

- Calculate the gradient of the loss function.
- Update each weight by taking a small step in the direction of the gradient (scaling the step by a parameter called the **learning rate**).

5.2.2 Gradient Descent Variations

Calculating the true gradient requires evaluating the entire dataset, which introduces key trade-offs:

- **Standard Gradient Descent:** Evaluates **all data points** to calculate the gradient before each weight update. While highly accurate, this is computationally expensive and slow for massive datasets.
- **Stochastic Gradient Descent (SGD):** Randomly selects **only one data point** at a time to estimate the gradient and update the weights. This is computationally rapid and scales well, but the gradient estimate is noisy and fluctuates.

- **Mini-batch Gradient Descent:** Groups the training data into small batches (e.g., 32 or 64 points). It computes the gradient and updates the weights based on each batch. This acts as a middle-ground approach, providing stable gradient estimates while maintaining computational efficiency.

5.3 Backpropagation

For simple networks, updating weights is straightforward. However, in networks with hidden layers, we only know the final output and the target label; we do not have target values for the hidden units. This is resolved using the **Backpropagation** algorithm.

Backpropagation Steps

1. Start with a random setting of weights.
2. Perform a **forward pass**: pass input data through the network, layer by layer, to calculate the final output.
3. Calculate the error (loss) at the output layer based on the true label.
4. Perform a **backward pass**: propagate the error **backwards** through the network, starting from the output layer back to the first hidden layer.
5. Estimate how much each individual hidden node contributed to the overall output error based on the strength of its connection weights.
6. Update all weights and biases in the network according to their calculated error contributions using gradient descent.

This iterative process is repeated over multiple **epochs** (passes through the entire training dataset) until the model converges.

5.4 Overfitting and Regularization (Dropout)

Because deep neural networks have thousands or millions of parameters, they are highly prone to **overfitting**—memorizing the training dataset’s noise and failing to generalize to unseen test data.

To combat overfitting, the lecture introduces **Dropout**:

- During training, a fixed percentage of units (e.g., 50%) are **temporarily and randomly removed (dropped out)** from the network during each weight update step.
- This forces the network to learn redundant, distributed representations, preventing it from over-relying on any single node and leading to a more robust, generalizable model.

6 Computer Vision and Convolutional Neural Networks

Computer Vision focuses on algorithms that allow computers to analyze and interpret digital images (e.g., handwriting recognition, self-driving car cameras, facial recognition).

6.1 Digital Image Representation

To a computer, an image is represented as a grid of pixels:

- **Grayscale:** Each pixel is represented by a single numerical value ranging from 0 (pure black) to 255 (pure white).
- **Color (RGB):** Each pixel is represented by three separate numerical channels: a Red channel, a Green channel, and a Blue channel, representing color intensity.

6.2 The Flattening Drawback

A naive approach to processing images with standard feed-forward networks involves "flattening" the pixel grid into a single 1D vector of inputs. This introduces two fatal flaws:

1. **Dimensionality Explosion:** A modest color image (e.g., 1000×1000 pixels) yields 3,000,000 input features, requiring an enormous number of weights and immense computational power.
2. **Loss of Spatial Context:** Flattening discards the spatial relationships between pixels (i.e., pixels adjacent horizontally or vertically). In computer vision, recognizing shapes, curves, and edges requires preserving local pixel structures.

6.3 Convolutional Neural Networks (CNNs)

A **Convolutional Neural Network (CNN)** is a specialized neural network structure that preserves spatial structure and drastically reduces parameter counts using convolution and pooling steps.

6.3.1 Image Convolution

Convolution involves sliding a small filter, known as a **kernel matrix** (typically 3×3 or 5×5), over the input image. For each position, the kernel's values are multiplied element-wise with the overlapping pixel values, and the results are summed up to produce a single value in a new **feature map**.

$$\text{Feature Map Value} = \sum (\text{Pixel Value} \times \text{Kernel Value}) \quad (6)$$

By choosing different kernels, we can filter the image to extract specific features:

- **Edge Detection Kernel:** A kernel with an 8 in the center and -1 in all surrounding neighboring cells:

$$\text{Kernel} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix} \quad (7)$$

If the center pixel is identical to all its neighbors, the sum is 0. If it differs significantly (such as at a boundary or edge), the sum is high. This extracts the structural outlines (edges) of the image, discarding irrelevant color details.

6.3.2 Pooling

Pooling reduces the dimensions of the feature maps by sampling from local pixel regions:

- **Max Pooling:** Slides a window (commonly 2×2) over the feature map and selects only the **maximum value** in that region.
- **Benefits:** Max pooling cuts the dimension of the data in half (reducing computational cost) and makes the model robust to slight shifts in the position of visual features (spatial invariance).
- **Average Pooling:** Extracts the average value of the region instead of the maximum.

6.3.3 The Complete CNN Pipeline

A typical deep CNN applies multiple alternating convolution and pooling steps to extract increasingly abstract features, followed by a final classification layer:

1. **Input Image:** The raw pixel grid (e.g., grayscale or RGB).
2. **Convolution + Pooling (Early Layers):** Extracts low-level features like edges, lines, and simple curves.
3. **Convolution + Pooling (Later Layers):** Combines low-level features into higher-level representations (e.g., eyes, noses, or numeric curves).
4. **Flattening:** Converts the final, highly-compact pooled feature maps into a flat 1D vector.
5. **Dense Layers:** Feeds the flat vector into a traditional deep feed-forward neural network.
6. **Output Layer:** Typically uses a **Softmax Activation Function** to output a probability distribution over the target classes (e.g., digits 0-9).

7 Recurrent Neural Networks (RNNs)

While CNNs excel at processing fixed-size spatial grids (images), many real-world datasets are structured as sequential inputs of varying lengths.

7.1 Feed-Forward vs. Recurrent Networks

- **Feed-Forward Neural Networks:** Data moves in a single direction (from inputs, through hidden layers, to outputs). They require inputs and outputs of a fixed, pre-determined size and process each data point independently, holding no memory of past inputs.
- **Recurrent Neural Networks (RNNs):** Feature connections that allow outputs to feed back into the network as inputs for future iterations. This feedback loop allows the network to maintain an internal **state (memory)** that persists across runs.

Maintaining a state is critical for understanding sequences of data where context matters (e.g., language or video frames).

7.2 Sequence-to-Sequence Relationships

RNN architectures are highly flexible and can map inputs and outputs of various structures:

1. **One-to-One:** A standard feed-forward task mapping a single input vector to a single output vector.
2. **One-to-Many:** Maps a single input to a sequence of outputs.
 - *Example: Image Captioning* (e.g., Microsoft's Caption Bot). The input is a single image, and the RNN sequentially outputs words of a descriptive sentence (e.g., "a group of people walking in front of the building") one token at a time, feeding its previous word back into itself to generate the next word.
3. **Many-to-One:** Maps a sequence of inputs to a single output.

- *Example 1: Sentiment Analysis.* A sequence of words in a movie review is processed step-by-step. The network updates its internal memory with each word, and outputs a single classification at the end (positive vs. negative).
- *Example 2: Video Classification.* Processes a sequence of video frames chronologically to classify the video's genre or detect violations.
- *Example 3: Voice Recognition.* Processes an audio waveform segment-by-segment to identify the speaker.

4. **Many-to-Many:** Maps a sequence of inputs to a sequence of outputs.

- *Example: Machine Translation* (e.g., Google Translate). A sequence of words in a source language is processed to construct an internal semantic encoding. This encoding is then used to sequentially generate the translated sentence in the target language. This is crucial because different languages utilize different word orders and lengths.

7.3 Long Short-Term Memory (LSTM)

Standard RNNs struggle to maintain memories over long sequences due to mathematical degradation. To solve this, a specialized RNN unit called the **Long Short-Term Memory (LSTM)** network is used, which utilizes special gate structures to preserve and retrieve information across very long sequence lengths.